

# Artur Pas

Rzeszów, Polska

+48 727 763 000

[artur.pas@proton.me](mailto:artur.pas@proton.me)

## O MNIE

Ukończyłem studia inżynierskie na Uniwersytecie Rzeszowskim na kierunku Informatyka oraz Technikum Informatyczne w Kolbuszowej. Poszukuję jakiejkolwiek pracy. Posiadam wysokie zdolności obsługi komputera i programowania ([Portfolio](#)), mam dobrą kondycję fizyczną, jestem punktualny, zorganizowany i szybko się uczę. Jestem otwarty na naukę nowych umiejętności oraz języków. W razie potrzeby mogę zmienić miejsce zamieszkania.

## UMIEJĘTNOŚCI

- JavaScript
- TypeScript
- HTML 5, CSS 3
- Python
- SQL
- Ruby
- Tailwind CSS
- React
- Vue
- Angular
- Laravel
- Ruby On Rails
- VS Code
- Git & GitHub
- DevTools
- PyCharm
- Figma & GIMP
- Postman, Swagger
- Windows, Linux
- PWA, RWD
- Rozwiązywanie problemów
- Bazy danych
- Angielski (B2)

## DOŚWIADCZENIE

### **Simplicity Games • Praktyki**

**Data:** 07.2024 – 08.2024

**Specjalizacja:** Unity Game Developer

### **Rubix Polska S.A. • Praktyki**

**Data:** 10.2018 – 11.2018

**Specjalizacja:** Pakowacz

## WYKSZTAŁCENIE

### **Uniwersytet Rzeszowski • Informatyka**

**Data:** 10.2021 – 03.2026

**Stopień:** Inżynier – Aplikacje Internetowe

### **Zespół Szkół Technicznych im. Bohaterów Września 1939 r w Kolbuszowej • Informatyka**

**Data:** 09.2016 – 06.2020

**Stopień:** Technik Informatyk

## CERTYFIKATY

**E.12** – Montaż i eksploatacja komputerów osobistych oraz urządzeń peryferyjnych

**E.13** – Projektowanie lokalnych sieci komputerowych i administrowanie sieciami

**E.14** – Tworzenie aplikacji internetowych i baz danych oraz administrowanie bazami

## [TuringMachineSimulator](#)

Zaawansowany, w pełni funkcjonalny symulator deterministycznej wielotaśmowej maszyny Turinga z graficzną wizualizacją funkcji przejścia, działający jako nowoczesna progresywna aplikacja internetowa (PWA) napisana w *HTML*, *CSS*, *JavaScript*, *TypeScript* z wykorzystaniem *Vite*, *React 19* i *PostCSS*.

- Opracowałem przejrzysty, responsywny interfejs z przyciąganiem widoku do wyznaczonych obszarów, pozwalający na łatwy dostęp do danych oraz zmianę motywu aplikacji.
- Wyodrębniłem oraz zintegrowałem z aplikacją fragment otwarto-źródłowej biblioteki *Neo4j Browser*, odpowiedzialny za wizualizację grafów, aby umożliwić użytkownikowi graficzne i interaktywne analizowanie stanów oraz przejść wizualizowanej maszyny Turinga.
- Zaprojektowałem oraz zaimplementowałem innowacyjny system szablonów składni programów maszyn Turinga, pozwalający na dynamiczną konfigurację leksera i parsera, przełamując problem sztywności notacji i oferując wykorzystanie stworzonego symulatora jako warstwy kompatybilności pomiędzy różnymi symulatorami poprzez transpilację programów.
- Napisałem własny lekser w oparciu o bibliotekę *CodeMirror* umożliwiający kolorowanie składni oraz parser przetwarzający kod programu maszyny Turinga do zoptymalizowanej postaci tablicy przejść stanów.
- Stworzyłem wydajne, asynchroniczne środowisko wykonawcze działające na *WebWorkerach*, operujące na dedykowanej strukturze danych do przechowywania wielomilionowych taśm. Taśmy te synchronizowane są między wątkami za pomocą bazy danych *IndexedDB*. Dzięki tym zabiegom wykonanie programu jest nawet tysiące razy szybsze niż w innych symulatorach, a wizualizacja nie jest obciążona kosztami obliczeń następnych kroków, które zawczasu wykonywane są przez osobne wątki.
- Rozszerzyłem kontrolę symulacji o warunkowe punkty przerywania, które w połączeniu z bogatą wizualizacją nadają aplikacji funkcjonalności debugera.
- Zebrałem przykładowe programy z symulatorów dostępnych w Internecie, tworząc bazę sprawdzonych programów do nauki oraz wykorzystałem biblioteki *ZenFS* oraz *fflate* do utworzenia systemu plików wraz z importem i eksportem umożliwiając użytkownikowi zbudowanie własnej bazy programów.

## [ConnectionGames](#)

Zbiór gier logicznych o tematyce łączenia. Celem gier jest odtworzenie poprawnych połączeń między obiektami na siatce. Gra oferuje 3 rodzaje gier w 3 poziomach trudności. Aplikacja została stworzona w *HTML*, *CSS*, *JavaScript*, *TypeScript* z wykorzystaniem *Vite*, *Vue 3* oraz *Pinia*.

- Stworzyłem estetyczny, responsywny interfejs z autorską animacją tła oraz animacją wygranej z użyciem biblioteki *canvas-confetti*.
- Zaimplementowałem system logiki rozgrywki dla 3 rodzajów gier logicznych opartych na *HTML* i *CSS* z obsługą urządzeń dotykowych oraz dynamicznie dostosowującym się rozmiarem, przy zachowaniu precyzji wyświetlania interfejsu.
- Opracowałem sposób zapisu, serializacji i deserializacji poziomów gier, pozwalający na przechowywanie niewielkiej ilości danych przy zachowaniu dobrej szybkości wczytywania poziomów.
- Utworzyłem edytor poziomów, a następnie zaprojektowałem 450 poziomów o rosnącym poziomie skomplikowania, po 50 na każdy poziom trudności danego trybu gry.

## [MatchstickEquations](#)

Gra logiczna polegająca na układaniu równań z zapalkami. Wykonana w dwóch wersjach: z użyciem języka C w terminalu oraz w języku C++ na *Arduino Yún* z wykorzystaniem matrycy punktowej *LED MAX7219 32x8*.

- Wymyśliłem algorytm generujący niepoprawne równania, które można poprawić jednym ruchem zapalki.
- Zaprojektowałem sposób reprezentacji liczb i znaków równania oraz sposób ich wyświetlania na ekranie.
- Zaimplementowałem od zera logikę oraz reguły gry, umożliwiając graczowi ciągłą rozgrywkę.

[ZOBACZ WIĘCEJ PROJEKTÓW](#)